ЛЕКЦИЯ 1 · 2026

Vite, Vite+, Oxc и Nginx

Современный инструментарий сборки,
линтинга и доставки фронтенда

143 карточки · 3 блока + Nginx

Vite · Vite+ · Oxc · Nginx

СОДЕРЖАНИЕ

О лекции

85
карточек

Vite

Создание проекта, конфиг, dev-сервер, env, ассеты

46
карточек

Vite+

vp: зависимости, dev, check, lint, fmt, test, CI

12
карточек

Oxc

Oxlint и Oxfmt: быстрый линт и формат на Rust

13
тем

Nginx

Конфиг, SPA, proxy, кэш, HTTPS

Все 143 карточки + блок Nginx: конфиг, SPA-fallback, proxy, кэш и HTTPS.

01

Vite

Сборка и dev-сервер · 85 карточек · 10 тем

Создание проекта

Два способа начать: новый проект или интеграция в готовый.

- npm create vite@latest — создать новый проект из шаблона.
- npm install -D vite — добавить Vite в существующий проект.
- Стартовые шаблоны: react, react-ts, vue, svelte, solid, preact, lit, vanilla.
- Vite — dev-сервер и сборщик: в dev отдаёт ESM по запросу, в build собирает бандл.

vite.config.ts: файл и форма

- Файл конфига для TypeScript-проекта — vite.config.ts.
- Минимальный конфиг: defineConfig({}) из пакета vite.
- Если конфиг зависит от команды — экспортируется функция: defineConfig(({ command }) => ({ ... })).
- command = serve при запуске vite (dev-режим).
- command = build при запуске vite build.
- process.env в Node.js — глобальный объект с переменными окружения процесса.

```
import { defineConfig } from 'vite';

export default defineConfig({});
```

Конфиг и .env: loadEnv

- .env недоступны в vite.config.ts через process.env: Vite откладывает их загрузку.
- Набор .env-файлов зависит от самого конфига — поэтому загрузка происходит позже.
- В process.env внутри конфига — только переменные окружения операционной системы.
- loadEnv вручную загружает .env: const env = loadEnv(mode, process.cwd(), '').
- process.cwd() — текущий рабочий каталог процесса Node.js.

```
export default defineConfig(({ command, mode }) => {
  const env = loadEnv(mode, process.cwd(), '');
  return {
    define: { __API__: JSON.stringify(env.VITE_API_URL) },
  };
});
```

Опции root и base

- root — папка, в которой находится файл index.html.
- По умолчанию root = process.cwd() — каталог запуска команды.
- base — запуск приложения при деплое в подпапку, а не в корень домена.
- По умолчанию base = '/'.
- В сборке base подставляет префикс во все пути и задаёт import.meta.env.BASE_URL.
- В dev-сервере приложение поднимается по этому пути: localhost:5173/app/.

Опция define

- define заменяет идентификаторы в коде на этапе сборки — аналог #define в C.
- В dev-режиме define добавляет свойство в globalThis.
- В сборке define меняет код на уровне AST, подставляя значение вместо идентификатора.
- Подводный камень: строки нужно оборачивать в JSON.stringify — значения подставляются как есть.
- Глобалы из define типизируют в vite-env.d.ts.

```
define: { __APP_VERSION__: JSON.stringify('1.0.0') }
```

Опция plugins

- plugins — массив плагинов Vite.
- false / null / undefined в массиве игнорируются — удобно для isDev && plugin().
- Вложенные массивы распаковываются.
- Promise дожидается разрешения перед запуском.

```
export default defineConfig({
  plugins: [react(), isDev && visualizer()],
});
```

Опция publicDir

- publicDir — папка статичных ассетов, которые отдаются как есть (favicon и т.п.).
- По умолчанию publicDir = public.
- В dev-режиме файлы доступны по корню /.
- В сборке папка копируется в корень outDir (dist).
- publicDir: false отключает обслуживание и копирование статической папки.

Опции resolve и clearScreen

- resolve.extensions позволяет опускать расширения файлов в операторах import.
- По умолчанию: ['.mjs', '.js', '.mts', '.ts', '.jsx', '.tsx', '.json'].
- Если подходит несколько файлов — побеждает расширение, стоящее раньше в списке.
- resolve.tsconfigPaths включает алиасы из compilerOptions.paths в tsconfig.json.
- По умолчанию resolve.tsconfigPaths = false.
- clearScreen включает очистку терминала при логах и пересборках.
- По умолчанию clearScreen = true; отключают, чтобы не терять историю логов при HMR.

Опции envDir и envPrefix

- envDir — папка, из которой Vite грузит .env-файлы.
- По умолчанию envDir = root (корень проекта).
- envDir: false отключает загрузку .env.
- envPrefix — префикс переменных, доступных в клиенте через import.meta.env.
- По умолчанию envPrefix = 'VITE_'.
- Меняют envPrefix, чтобы защитить серверные секреты от попадания в клиентский бандл.

Dev-сервер: порт и HTTPS

- server.port по умолчанию 5173.
- server.strictPort запрещает занимать другой порт: если указанный занят, сервер не запустится.
- server.https включает TLS/HTTP2 для dev-сервера.
- Принимает { key, cert } или плагин @vitejs/plugin-basic-ssl.
- @vitejs/plugin-basic-ssl: добавь basicSsl() в plugins.

```
import basicSsl from '@vitejs/plugin-basic-ssl';

export default defineConfig({ plugins: [basicSsl()] });
```

Dev-сервер: браузер и CORS

- server.open открывает приложение в браузере при старте dev-сервера.
- Строковое значение server.open — путь (pathname), который нужно открыть.
- server.cors настраивает CORS для dev-сервера.
- Значения server.cors: true (любой origin) или объект настроек, как CORS-middleware.

Dev-сервер: proxy и консоль

- server.proxy перенаправляет запросы dev-сервера на другой сервер (бэкенд), обходя CORS.
- server.forwardConsole перенаправляет события консоли браузера в терминал Vite.
- server.forwardConsole: true включает перенаправление.
- Полезен при разработке с AI-агентами — им становятся видны ошибки клиента.

```
server: {
  proxy: {
    '/api': { target: 'http://localhost:8080', changeOrigin: true, ws: true },
  },
},
```

Плагины и кеш зависимостей

- Плагин Vite для React — @vitejs/plugin-react.
- @vitejs/plugin-basic-ssl подключается как basicSsl() в plugins.
- Vite кеширует предварительно упакованные зависимости в node_modules/.vite.

```
import react from '@vitejs/plugin-react';
import basicSsl from '@vitejs/plugin-basic-ssl';

export default defineConfig({
  plugins: [react(), basicSsl()],
});
```

Ассеты и препроцессоры

- import imgUrl from './img.png' при сборке превращается в URL-адрес файла.
- imgUrl используется как обычный URL — например, в атрибуте src тега <img>.
- Плагины для .scss / .sass / .less / .styl не нужны — поддержка встроена.
- Нужен сам препроцессор как dev-зависимость: npm install -D sass.
- @import для Sass/Less учитывает алиасы, настроенные в Vite.

```
import imgUrl from './img.png';

<img src={imgUrl} alt="" />
```

env: префикс и доступ

- Переменные в .env попадают в клиентский код, только если начинаются с VITE_.
- В import.meta.env переменные всегда string — даже если в .env записано число или булево.
- Сравнение с булевым — явное: import.meta.env.VITE_F === 'true'.
- В клиенте переменные доступны через import.meta.env.
- import.meta.env.DB_PASSWORD без префикса VITE_ вернёт undefined.

> Секреты без префикса VITE_ не попадают в бандл. Это не замена серверной авторизации, но защищает от случайной утечки.

env: порядок загрузки и встроенные поля

- Порядок приоритета: .env.[mode].local → .env.[mode] → .env.local → .env.
- MODE — текущий режим: development или production.
- BASE_URL — базовый URL приложения (из опции base).
- PROD = true в продакшен-сборке.
- DEV = true в режиме разработки.
- SSR = true при серверном рендеринге.

```
import.meta.env.MODE      // 'development'
import.meta.env.BASE_URL  // '/'
import.meta.env.PROD      // false
import.meta.env.DEV       // true
```

env: import.meta и типы значений

- Кастомные переменные — всегда string; встроенные PROD / DEV / SSR — boolean.
- import.meta — объект с метаданными текущего ESM-модуля.
- import.meta.env — объект с доступными переменными окружения, который создаёт Vite.

```
import.meta.env.MODE      // string
import.meta.env.VITE_API  // string
import.meta.env.PROD      // boolean
```

> Vite подставляет значения в код на этапе сборки — поэтому import.meta.env работает и в браузере.

env: типизация в TypeScript

- ImportMetaEnv — интерфейс, описывающий структуру объекта import.meta.env.
- TypeScript по умолчанию знает только встроенные поля: MODE, DEV, PROD, BASE_URL, SSR.
- import.meta.env.VITE_APP_TITLE без типизации — ошибка типа.
- Кастомные переменные типизируют в src/vite-env.d.ts.
- Расширяют интерфейсы ImportMetaEnv и ImportMeta.
- /// <reference types="vite/client" /> подключает типы встроенных import.meta.env и ассетов.

```
interface ImportMetaEnv {
  readonly VITE_APP_TITLE: string;
}
interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

02

Vite+

Единый CLI для установки, проверки и сборки · 46 карточек · 13 тем

Vite+: состав и установка зависимостей

- Vite+ состоит из глобального CLI vp и локального пакета vite-plus в каждом проекте.
- vp add <pkg> — добавить пакет.
- vp add -D <pkg> — добавить dev-пакет.
- vp remove <pkg> — удалить пакет.
- vp update — обновить зависимости.
- devEngines.packageManager в package.json задаёт менеджер пакетов для проекта.
- packageManager в package.json указывает точную версию менеджера, например bun@1.1.20.

```
"devEngines": { "packageManager": "bun" },
"packageManager": "bun@1.1.20"
```

Vite+ dev и check

- vp dev запускает dev-сервер.
- Конфигурация Vite+ находится в стандартном файле vite.config.ts.
- vp check объединяет форматирование, линтинг и проверку типов.
- vp check --fix включает автоисправление.

> Один конфиг vite.config.ts для dev, lint, fmt, test и run — не нужно держать отдельные oxlint.config.ts, oxfmt.config.ts и vitest.config.ts.

Vite+ lint

- vp lint запускает линтинг.
- vp lint --fix включает автоисправление.
- typeCheck запускает компилятор TypeScript во время линтинга.
- typeAware ищет логические ошибки на основе типов.
- vp lint основан на Oxlint.
- Конфиг vp lint размещают в vite.config.ts, в блоке lint.
- Не стоит размещать конфиг в oxlint.config.ts.
- Базовая конфигурация: lint: { ignorePatterns: ['dist/**'] }.

```
export default defineConfig({
  lint: { ignorePatterns: ['dist/**'] },
});
```

Vite+ format и test

- vp fmt форматирует код.
- vp fmt . --write записывает отформатированный код в файлы.
- vp fmt основан на Oxfmt.
- Конфиг vp fmt — в vite.config.ts, блок fmt.
- Не в oxfmt.config.ts.

- vp test запускает тесты.
- Тесты Vite+ основаны на Vitest.
- Конфиг vp test — в vite.config.ts, блок test.
- Не в vitest.config.ts.

Vite+ run: скрипты и задачи

- vp run <script-name> запускает скрипты из package.json.
- vp dev — встроенная команда, а vp run dev запускает скрипт dev из package.json.
- Задача в конфиге: блок run.tasks в vite.config.ts.
- dependsOn указывает зависимость одной задачи от другой.
- vp run <task-name> запускает задачу из конфига.
- Задача (task) — именованный скрипт, состояние которого Vite+ кеширует по входным и выходным файлам.
- Пример: build: { command: 'vp build', dependsOn: ['lint'] }.

```
run: {
  tasks: {
    build: { command: 'vp build', dependsOn: ['lint'] },
  },
},
```

vpx, build, upgrade и CI

- vpx — аналог npx / bunx в Vite+.
- vp build собирает проект.
- vp upgrade обновляет глобальный vp.
- Для CI используют voidzero-dev/setup-vp.

> Vite+ закрывает весь цикл: install → dev → check → lint → fmt → test → run → build → CI.

Vite+ commit hooks и staged

- vp hooks enable — включить Git-хуки.
- vp hooks disable — выключить Git-хуки.
- vp hooks status — проверить статус Git-хуков.
- Скрипты Git-хуков лежат в папке .vite-hooks в корне проекта.
- Папку .vite-hooks нужно коммитить, чтобы хуки были одинаковыми у всех.
- Хук pre-commit: создать .vite-hooks/pre-commit с командой vp staged.
- vp staged проверяет только изменённые файлы перед коммитом.
- Настройка staged в vite.config.ts: { '*.{js,ts,tsx}': 'vp check --fix' }.

```
staged: { '*.{js,ts,tsx}': 'vp check --fix' }
```

03

Oxc

Oxlint и Oxfmt: быстрый линт и форматирование на Rust · 12 карточек · 5 тем

Oxc и Oxlint

- Oxc — набор высокопроизводительных инструментов для JavaScript и TypeScript на Rust.
- Oxlint — высокопроизводительный линтер для JS/TS на стеке Oxc, совместимый с ESLint.
- Oxlint использует tsgo — он же TypeScript 7.
- Установка: npm install -D oxlint.
- Скрипты: "lint": "oxlint" и "lint:fix": "oxlint --fix".

```
"scripts": {
  "lint": "oxlint",
  "lint:fix": "oxlint --fix"
}
```

Oxlint: конфигурация

- oxlint --init инициализирует конфиг.
- Файл конфига: .oxlintrc.json или oxlint.config.ts.
- Базовый конфиг: import { defineConfig } from 'oxlint'; export default defineConfig({});

```
import { defineConfig } from 'oxlint';

export default defineConfig({});
```

Oxlint: встроенные плагины

- Плагин — именованная группа правил; его включение делает правила доступными.
- Встроенные плагины включают в поле plugins, например plugins: ['import'].
- Запись plugins переопределяет плагины, включённые по умолчанию.
- Плагин eslint включён по умолчанию — он подключает основные правила ESLint.

```
export default defineConfig({
  plugins: ['import'],
});
```

Шпаргалка: команды

Vite

```
npm create vite@latest
npm install -D vite
vite
vite build
```

Vite+

```
vp add / add -D
vp remove
vp update
vp dev / build / test
vp check / lint / fmt
vp run <script>
vpx
vp hooks enable
vp staged
```

Oxc

```
npm install -D oxlint
oxlint
oxlint --fix
oxlint --init
```

04

Nginx

Конфиг, раздача SPA, proxy и HTTPS · основные настройки

Зачем Nginx фронтенду

- Отдаёт статику (HTML, JS, CSS, шрифты) быстрее и дешевле, чем Node-сервер.
- Reverse proxy: прячет бэкенд, снимает CORS и терминирует TLS.
- Сжимает ответы (gzip) и кэширует статику на клиенте.
- Держит много соединений за счёт event-based модели: master + worker.
- Балансирует нагрузку между несколькими инстансами приложения.

> В типичном деплое Vite-приложения Nginx — точка входа, а Node/API — за ним.

Структура конфига

- Простые директивы заканчиваются ;, блочные — { ... }.
- Контексты: main → events → http → server → location.
- server слушает порт и домен, location выбирает правило по URI.
- Комментарии — #.
- Подключение файлов: include /etc/nginx/conf.d/*.conf;.
- Основной файл: /etc/nginx/nginx.conf.

```
events { worker_connections 1024; }

http {
  include      mime.types;
  default_type application/octet-stream;

  server {
    listen 80;
    server_name example.com;
    location / { ... }
  }
}
```

Минимальный статический сервер

- listen 80; — порт (и адрес).
- server_name example.com; — домен; _ — любой.
- root /usr/share/nginx/html; — корень файлов.
- index index.html; — файл по умолчанию для /.
- include mime.types; — правильные Content-Type.
- sendfile on; — эффективная отдача файлов.

```
server {
  listen 80;
  server_name example.com;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ =404;
  }
}
```

Раздача SPA: try_files

- SPA — один index.html, маршруты обрабатывает клиент.
- try_files $uri $uri/ /index.html; — ищет файл, затем папку, затем отдаёт index.
- Без fallback прямой заход на /posts/1 даст 404.
- try_files $uri $uri/ =404; — обычная статика.
- error_page 404 /index.html; — альтернативный fallback.

```
location / {
  try_files $uri $uri/ /index.html;
}
```

Порядок важен: сначала реальные файлы (JS/CSS), потом fallback на index.html.

location: приоритеты

- location = /path — точное совпадение, самый высокий приоритет.
- location ^~ /prefix/ — самый длинный prefix; регулярки не проверяются.
- location ~ regex — регулярка с учётом регистра; ~* — без.
- location /prefix/ — обычный prefix; побеждает самый длинный.
- Порядок: = → самый длинный prefix → ^~ → regex по порядку → prefix.

```
location = / { ... }            # точное
location ^~ /assets/ { ... }    # prefix, без regex
location ~* \.(js|css)$ { ... } # regex
location / { ... }              # общий prefix
```

Regex проверяются в порядке записи — первый матч выигрывает.

Reverse proxy: proxy_pass

- proxy_pass http://backend:8080; — передать запрос на бэкенд.
- Слэш на конце proxy_pass .../ срезает префикс location; без слэша — URI как есть.
- proxy_set_header Host $host; — сохранить исходный Host.
- X-Real-IP, X-Forwarded-For, X-Forwarded-Proto — реальный клиент и схема.
- proxy_http_version 1.1; + Connection "" — keep-alive до бэкенда.
- Таймауты: proxy_connect_timeout, proxy_read_timeout.

```
location /api/ {
  proxy_pass http://backend:8080/;
  proxy_http_version 1.1;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header X-Forwarded-Proto $scheme;
}
```

upstream и балансировка

- upstream backend { server ...; } — группа бэкендов.
- По умолчанию — round-robin.
- least_conn; — на менее загруженный.
- ip_hash; — привязка клиента к серверу (сессии).
- keepalive 32; — пул keep-alive соединений.
- max_fails / fail_timeout — пассивные проверки.

```
upstream backend {
  least_conn;
  server 10.0.0.1:3000 max_fails=3 fail_timeout=30s;
  server 10.0.0.2:3000;
  keepalive 32;
}
```

Кэширование и gzip

- expires 1y; — Expires + Cache-Control: max-age.
- add_header Cache-Control "public, immutable"; — для хэшированных ассетов.
- expires -1; / Cache-Control: no-cache — для index.html.
- gzip on; + gzip_types — сжатие JS/CSS/JSON/SVG.
- gzip_min_length, gzip_comp_level, gzip_vary on;.
- add_header ... always; — заголовок и на ошибках.

```
gzip on;
gzip_comp_level 5;
gzip_min_length 1024;
gzip_types text/css application/javascript application/json image/svg+xml;
gzip_vary on;

location ~* \.(js|css|woff2|png|svg)$ {
  expires 1y;
  add_header Cache-Control "public, immutable";
}
```

HTTPS и редирект с HTTP

- listen 443 ssl; — HTTPS-сервер.
- ssl_certificate / ssl_certificate_key — цепочка и ключ.
- ssl_protocols TLSv1.2 TLSv1.3; — только современные версии.
- http2 on; — HTTP/2 (в новых версиях отдельной директивой).
- Редирект с 80 на 443: return 301 https://$host$request_uri;.
- ssl_session_cache shared:SSL:10m; — переиспользование сессий.

```
server {
  listen 80;
  server_name example.com;
  return 301 https://$host$request_uri;
}
```

```
server {
  listen 443 ssl;
  http2 on;
  ssl_certificate     /etc/nginx/ssl/fullchain.pem;
  ssl_certificate_key /etc/nginx/ssl/privkey.pem;
  ssl_protocols       TLSv1.2 TLSv1.3;
}
```

Безопасность и лимиты

- client_max_body_size 10m; — лимит тела запроса (413 при превышении).
- server_tokens off; — не раскрывать версию Nginx.
- add_header X-Content-Type-Options nosniff always;.
- X-Frame-Options SAMEORIGIN / Referrer-Policy.
- limit_req_zone + limit_req — rate limiting.
- limit_conn — ограничение соединений.
- always у add_header — заголовки и на 4xx/5xx.

```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

location /api/ {
  limit_req zone=api burst=20 nodelay;
}
```

Логи и перезагрузка конфига

- access_log /var/log/nginx/access.log;.
- error_log /var/log/nginx/error.log warn;.
- log_format — свой формат (время, upstream, статус).
- nginx -t — проверить конфиг перед reload.
- nginx -s reload — плавно применить; -s quit — graceful stop.
- systemctl reload nginx — reload через systemd.

```
log_format main '$remote_addr - $status $request_time '
                '$request "$http_user_agent"';
access_log /var/log/nginx/access.log main;
```

Сначала nginx -t, потом nginx -s reload — иначе можно уронить прод.

Полный конфиг: Vite SPA + API

Один server: статика, SPA-fallback, кэш и proxy на API.

```
server {
  listen 80;
  server_name example.com;
  root /usr/share/nginx/html;
  index index.html;
  gzip on;
  gzip_types text/css application/javascript application/json;
  location / {
    try_files $uri $uri/ /index.html;
  }
  location ~* \.(js|css|woff2|png|svg)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
  }
  location /api/ {
    proxy_pass http://backend:8080/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Частые ошибки и проверка

- try_files без fallback → 404 на прямых ссылках SPA.
- Забытый слэш в proxy_pass → неверный путь на бэкенде.
- add_header без always → нет заголовков на ошибках.
- root вместо alias (или наоборот) → неверные пути.
- Кэширование index.html → пользователи видят старую версию.
- Нет nginx -t перед reload → риск уронить конфиг.

```
nginx -t            # проверить конфиг
nginx -s reload     # применить без простоя
```

> Симптом: 404 на /posts/1, старый index.html, пустой X-Forwarded-For — смотрите error.log.

Что дальше

- Практика: собрать проект на Vite+ с Oxlint и раздать его через Nginx.
- Развернуть SPA: Docker + Nginx, try_files, proxy на API.
- Настроить CI: voidzero-dev/setup-vp + vp check.
- Документация: vite.dev, viteplus.dev, oxc.rs, nginx.org.
- Карточки: 143 вопроса + конфиг Nginx — для самопроверки.

Покрытие: Vite — 85 карточек, Vite+ — 46, Oxc — 12. Плюс блок Nginx: конфиг, proxy, кэш, HTTPS.